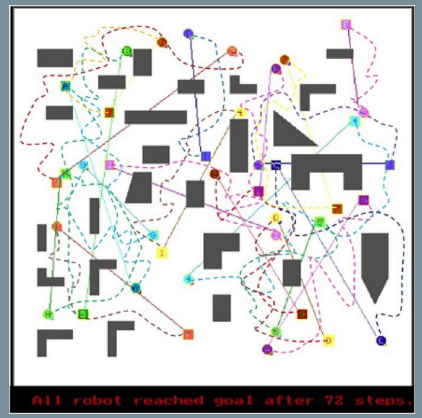
Path Planning Using Constrained Delaunay Triangulation

Keshawn B. & Nadezhda D.
Comp Geo Fall 2025

Motivation

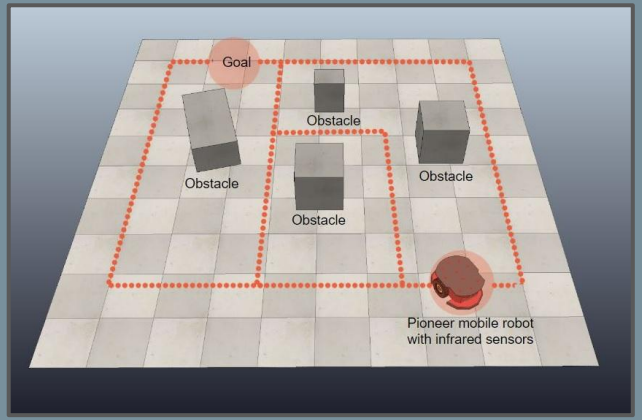
- Robotics is an ever-growing field of study
- The less user-robot interaction when executing commands, the better
- How could we make robots navigate a complex area with little human intervention?
    - Use Geometry!



Setting up path-finding problem

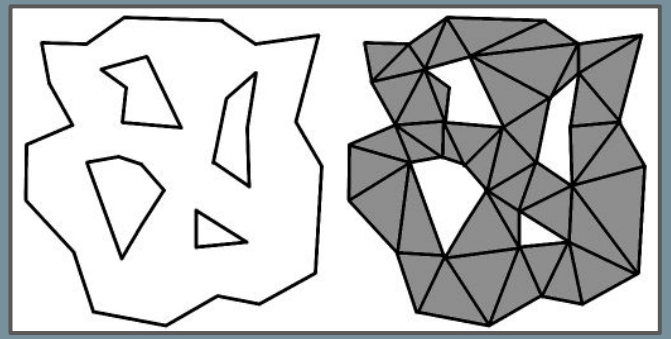
- Given:
  - Robot
  - Polygonal environment P, free space
  - Start point in P
  - End point in P

What would be the shortest path from point A to point B?



Polygonal Regions and Triangulations

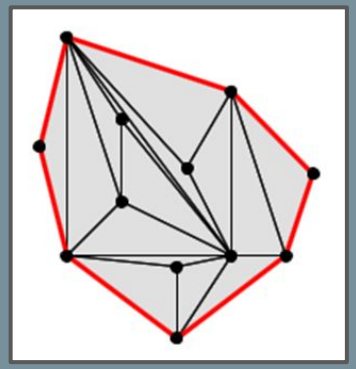
- Polygonal regions have the traits of the formal polygon, except they can contain polygonal holes.
- Polygonal regions with polygonal holes can be partitioned such that each sub-region is a polygon
- All polygons can be triangulated
- Therefore, polygonal regions can be triangulated!

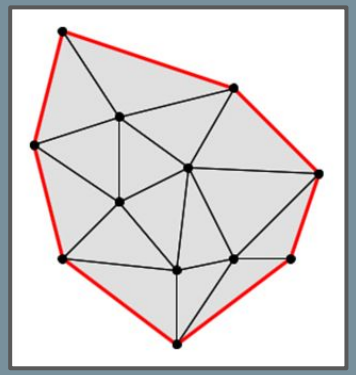


Delaunay Triangulation

DT

- Triangulation of a point set
- Empty Circumcircle property
- Avoids skinny triangles

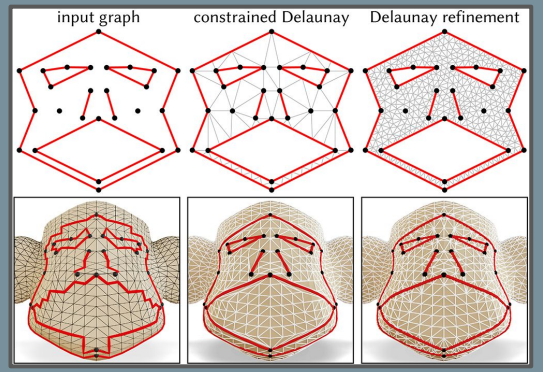




Constrained Delaunay Triangulation (CDT)

Triangulation of the vertices with the following properties:

1) The prescribed edges are included in the triangulation
2) It is as close as possible to a Delaunay Triangulation



Optimal Algorithm Time Complexity: O(nlogn)

Applications: Used in motion planning, GIS for terrain visualization, computer graphics, and creating triangular meshes

Algorithm for CDT Path-Finding

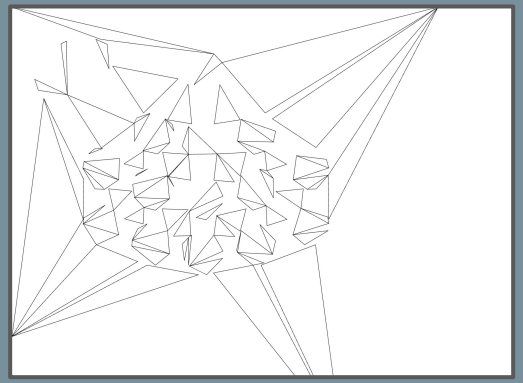
Input: Image of top view of classroom

1. Detect obstacles and draw polygons around them
2. Avoiding polygonal regions within, triangulate environment
3. For each triangle, compute and store centroid
4. Construct network of centroids such that each edge connecting two centroids does not intersect an internal polygonal region
5. Given the start and end centroid, implement Dijkstra's algorithm for shortest path with the network

Output: List of coordinates for (theoretical) robot to follow

CGAL Versus Shapely

- Originally attempt in CGAL C++
  - Complicated domain marking for CDT faces
  - Difficult to visualize
- Our Implementation: Shapely & Networkx
  - Accessible reconstruction of environment as polygons
  - Great visualization support for graphs



CGAL's Broken CDT

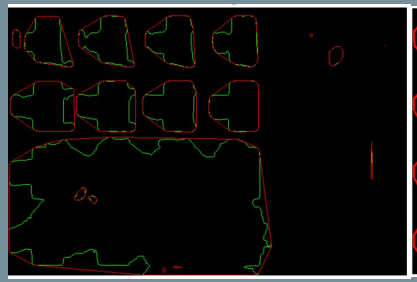
Detect Environment & Obstacle Polygons

Used Python's OpenCV

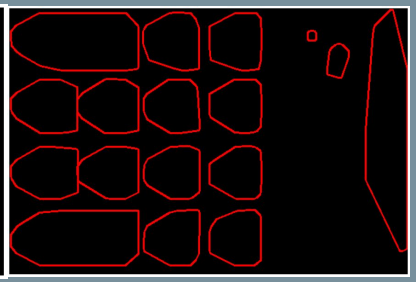
1. Inverted image
2. Blurred output of Step 1
3. Implemented Canny Edge detection on output of Step 2
4. Blurred output of Step 3
5. Created "contour" for each explicit edge from output of Step 4
6. Constructed Convex Hulls of selectively chosen contours. **This step actually makes hulls around the obstacles**



Original Image

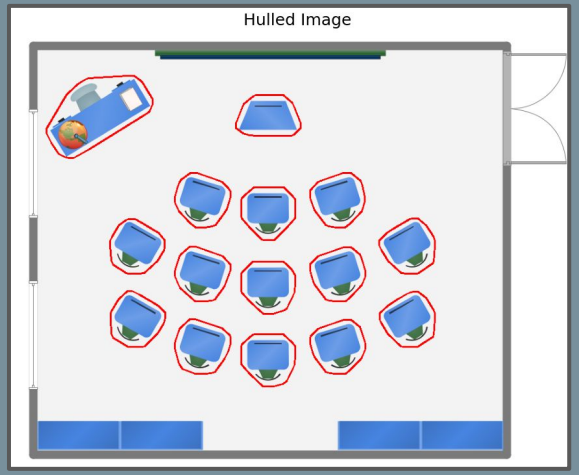


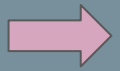
Thresholding

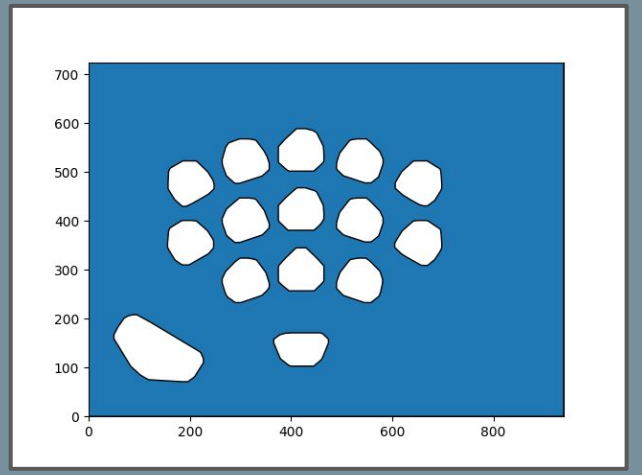


Canny Edge Detection

Detect Environment & Obstacle Polygons

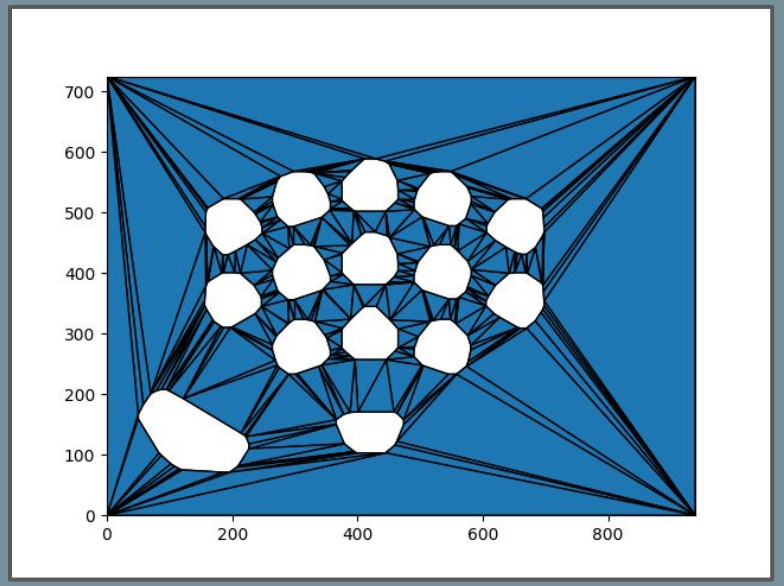






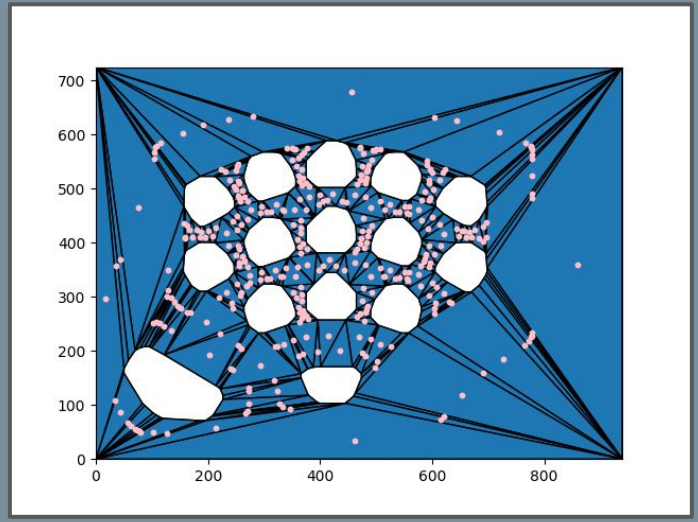
Create Constrained Delaunay Triangulation (CDT)

- CDT respects obstacle polygon edge constraints
- Triangulates the free space



Compute CDT Centroid

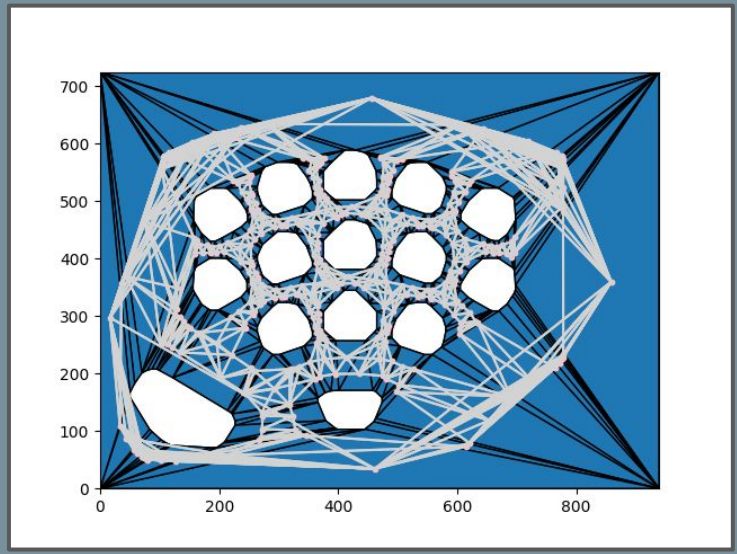
- Find centroids of each triangle of CDT using Shapely
- Each centroid becomes a navigable node



Build Centroid Graph

Construct the centroid graph, representing navigation graph:

- Nodes - CDT centroids
- Edges - Adjacent centroids connected, avoid obstacle edge intersections



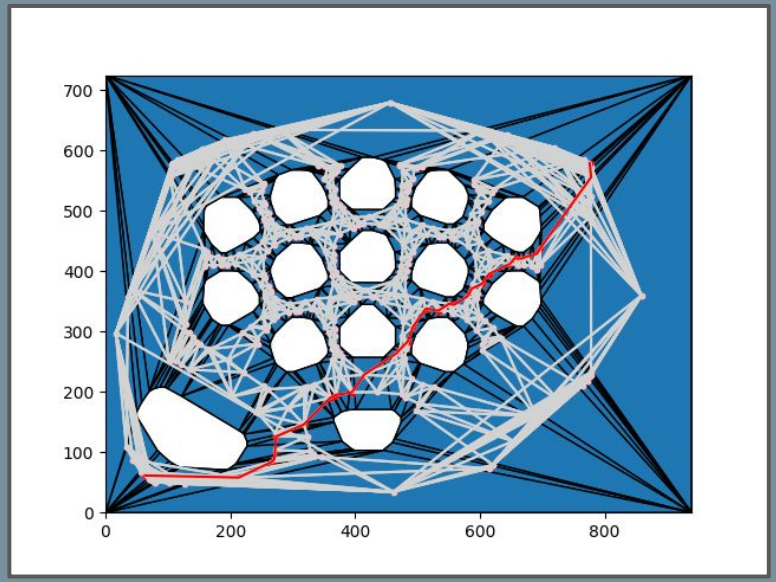
Path Finding Using Dijkstra's Algorithm

Path found via the following:

- Identify closest centroid to start & end points
- Compute shortest path with Dijkstra

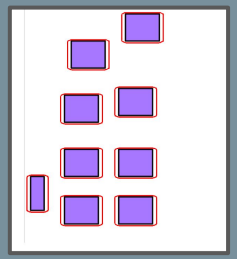
<u>Results:</u>
Number of Triangles in CDT: 344
Number of Centroids: 344

Number of Centroids in the Shortest Path: 33

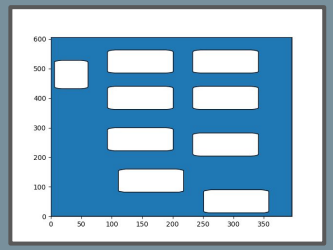


Our Classroom!

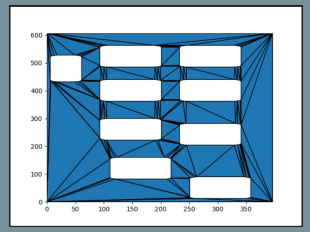
Initial Image



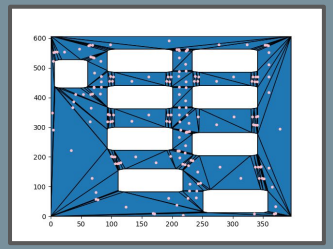
Setup



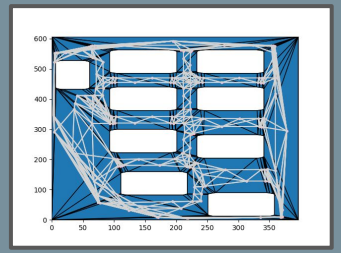
Triangulate



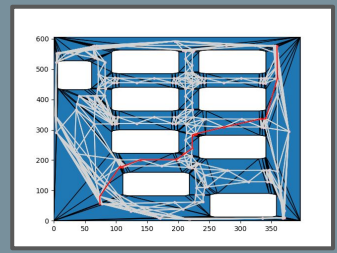
Centroids



Centroid Graph



Shortest Path



Takeaways

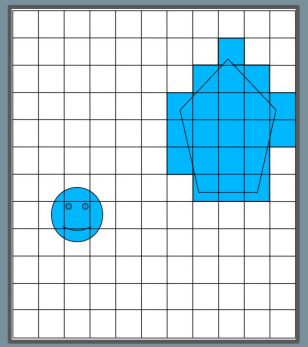
- Constrained Delaunay triangulation reduces search complexity and enables efficient path planning!
- Limitation in deploying a physical robot due to outdated hardware (Lego Robots)



Lego Robot

Future Work

- Implement physical robot to move throughout a classroom
- Use different shortest-path algorithms to compare efficiency of method
  - A*
  - Rapidly Exploring Random Tree (RRT)
  - Probabilistic RoadMap (PRM)
- Comparing map representations for best to use when path-finding
  - Regular vs Delaunay Triangulations
  - Maybe even grid representations



References

Github Repository: https://github.com/kboughto/CompGeom_FinalProject

Libraries Used:

- Python's OpenCV:
  - Finding contours properly: https://learnopencv.com/edge-detection-using-opencv/
  - Making convex hulls around images: https://learnopencv.com/convex-hull-using-opencv-in-python-and-c/
- Python's Shapely:
  - Delaunay Triangulations: https://shapely.readthedocs.io/en/2.1.2/reference/shapely.constrained_delaunay_triangles.html
  - Visualizing Shapely objects: https://stackoverflow.com/questions/8919719/how-to-plot-a-complex-polygon
- Python's Networkx:
  - Constructing graph: https://networkx.org/documentation/stable/tutorial.html

Paper for Algorithm Implemented:

Jan, Gene Eu, et al. "Obstacle-Avoidance Path Planning Based on Delaunay Triangulation." The 9th International Conference on Advanced Robotics and Intelligent Systems (ARIS'2021), Aug. 2021, www.researchgate.net/publication/357683816_Obstacle-Avoidance_Path_Planning_Based_on_Delaunay_Triangulation. Accessed 6 Nov. 2025.

Thank you!